New Paradigms for KV Cache Reuse

Xing Ma

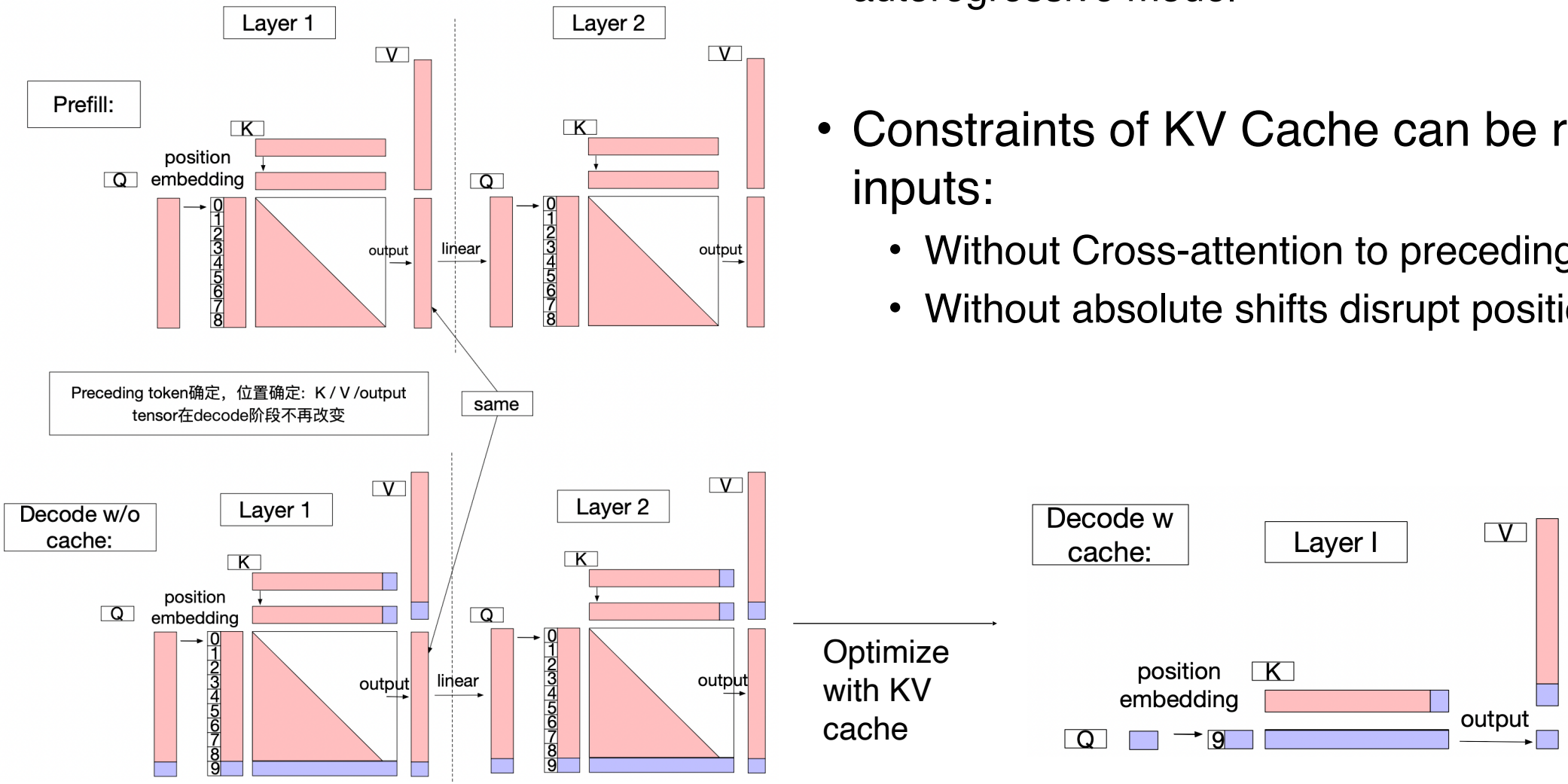
2026-5-15

Outline

- Background & motivation
- Position-Dependent Caching (PDC)
  - Prefix cache(vllm, sglang)
- Position-Independent Caching (PIC)
  - CacheBlend(EuroSys’25)
- Relative-Position-Dependent Caching(RPDC)
  - CacheSlide(FAST’26)
- Summary

Background

Why KV cache?

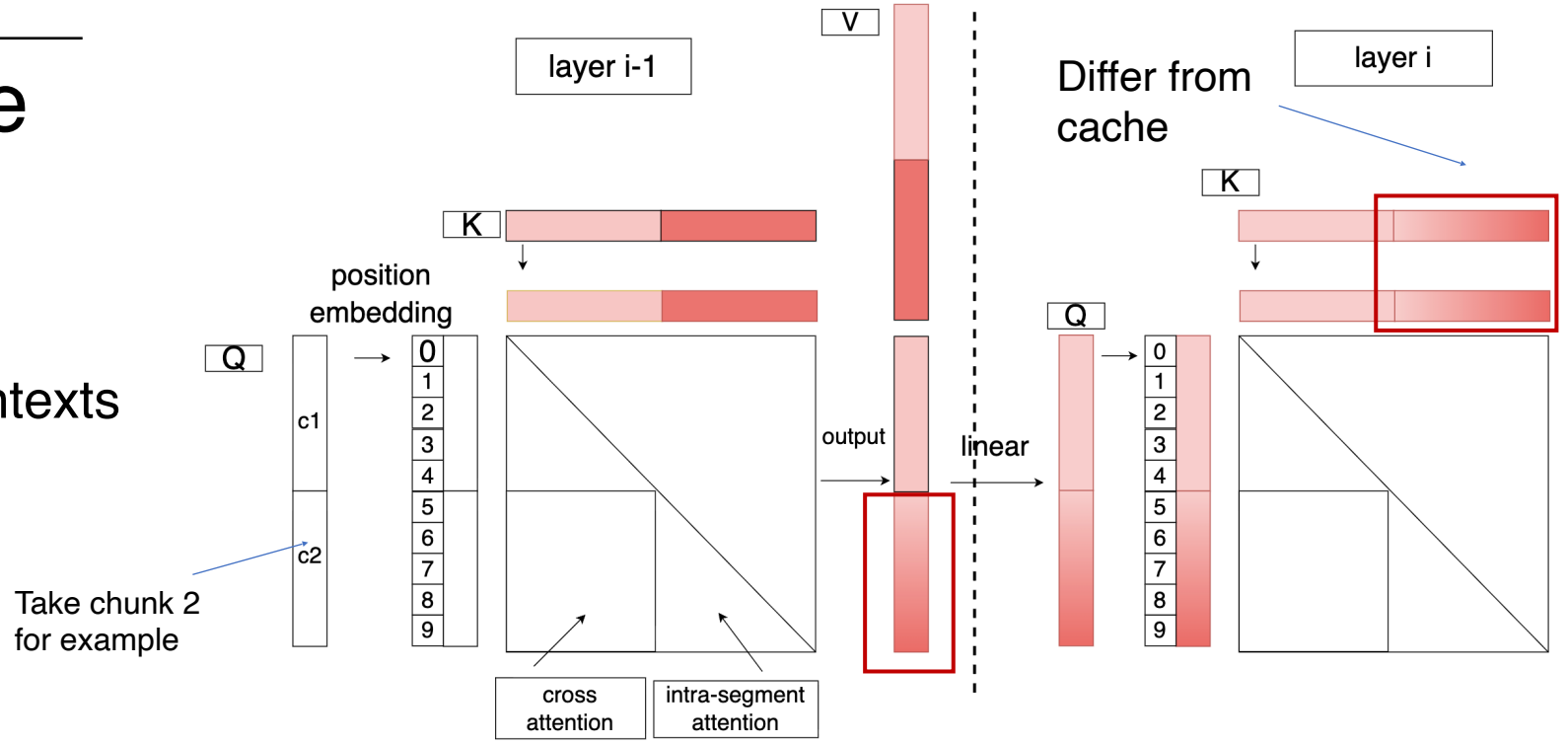


- KV Cache: KV tensor of prompt tokens can be reuse in autoregressive model
- Constraints of KV Cache can be reuse across inputs:
  - Without Cross-attention to preceding contexts
  - Without absolute shifts disrupt positional encoding

Background

Example of cache failure

- constraints of KV cache reuse across requests:
  - Cross-attention to preceding contexts
  - Absolute shifts disrupt positional encoding.



- **Consequences of these constraints:**
  - Strict Prefix-Ordering: Computation demands exact prefix matching.
  - Poor Cache Reuse even **fixed segments across requests.**

Background

Three KV cache reuse Paradigms in LLM serving

- (1) Position-Dependent Caching： require exact position alignment
  - Prefix Cache in vllm/Sglang

- (2) Position-Independent Caching： modular reuse of KV vectors regardless of prefixes
  - CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion(EuroSys'25)

- (3) Relative-Position-Dependent Caching：Constant relative positions in immutable segments
  - CacheSlide: Unlocking Cross Position-Aware KV Cache Reuse for Accelerating LLM Serving (FAST'26)

- Target: 希望通过算法上的修正来复用static segment的kv cache, 让LLM从以位置为强约束的串行计算，演变成可支持以模块为中心的可复用，可调度的灵活计算范式

EuroSys 25 best paper

CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion

Jiayi Yao
University of Chicago/CUHK Shenzhen

Hanchen Li
University of Chicago

Yuhan Liu
University of Chicago

Siddhant Ray
University of Chicago

Yihua Cheng
University of Chicago

Qizheng Zhang
Stanford University

Kuntai Du
University of Chicago

Shan Lu
Microsoft Research / University of Chicago

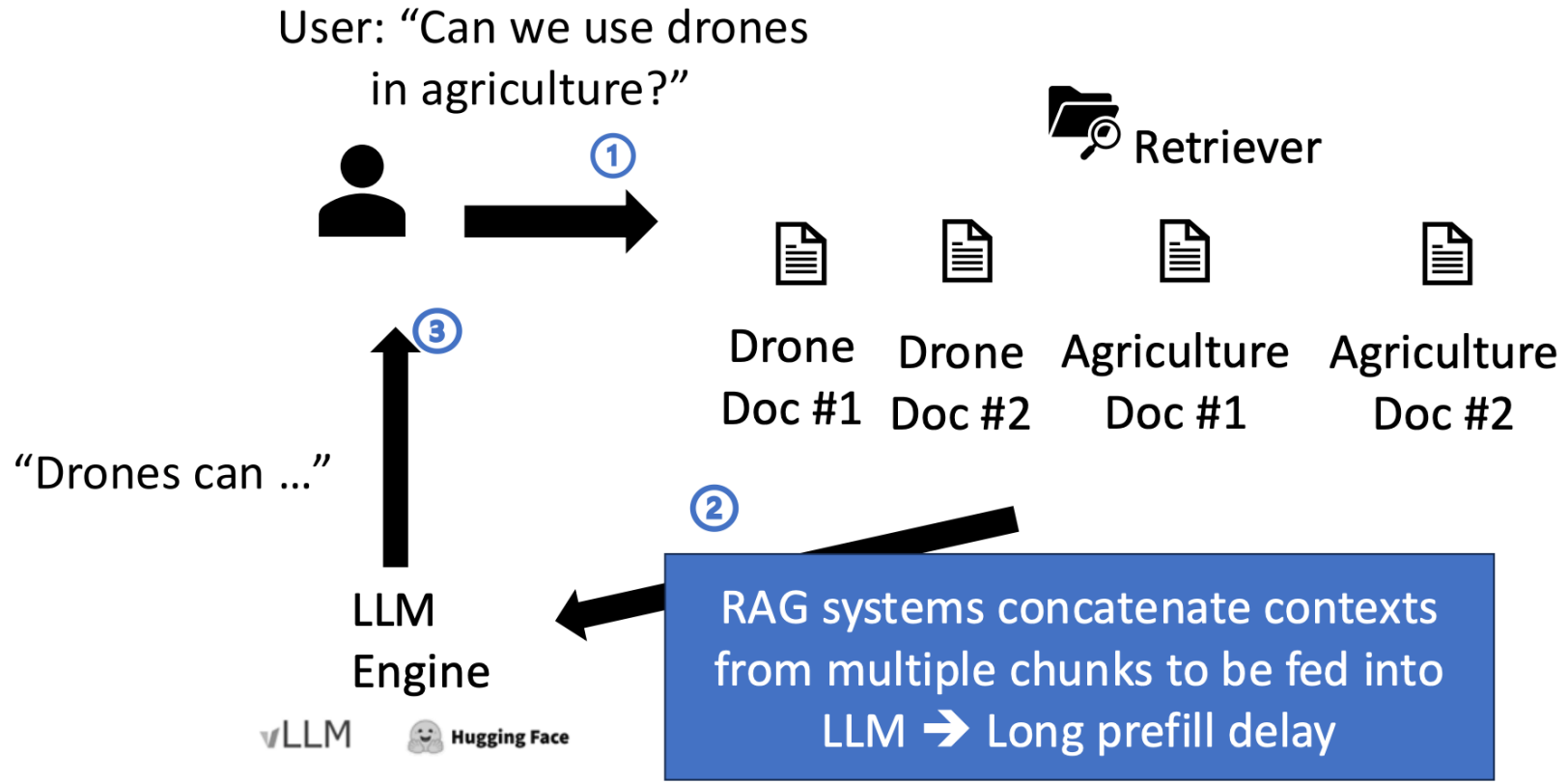
Junchen Jiang
University of Chicago

- TL;DR:
  - Target scenario: RAG
  - Core idea: blending different KV cache for different chunks in RAG for reducing inference latency
  - How to recover acc: recompute subset

Background

Challenge in RAG Systems

- **Scenario:** RAG (Multiple reusable text segments)
- **Prompt Pattern:** [Doc 1] + [Doc 2] + [Doc 3] + [Query]



Background

Limitations of existing solution

- **Prefix Caching:** Reuses only the initial segment -> **Limited performance gains**

- **Full KV Reuse:** Ignores cross-attention dependencies -> bad generation quality

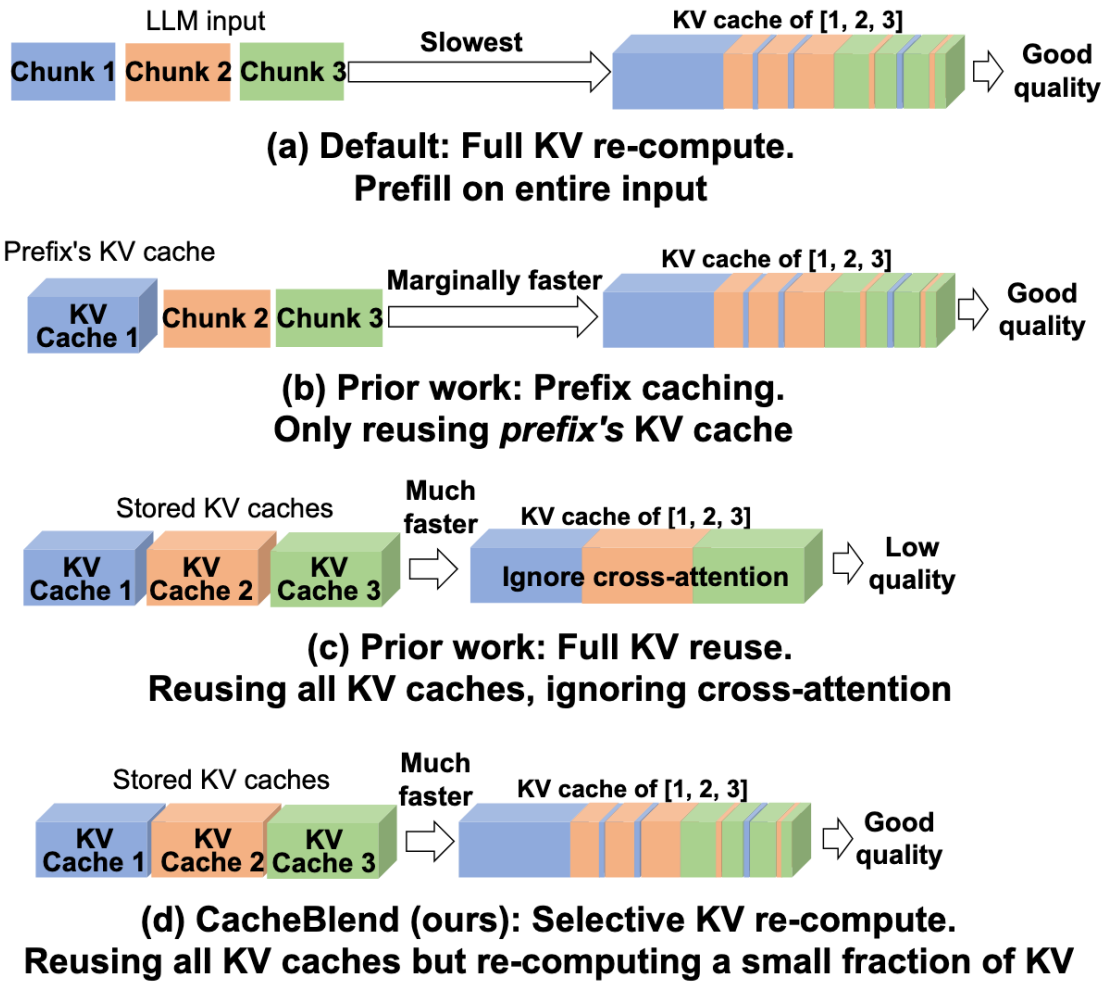
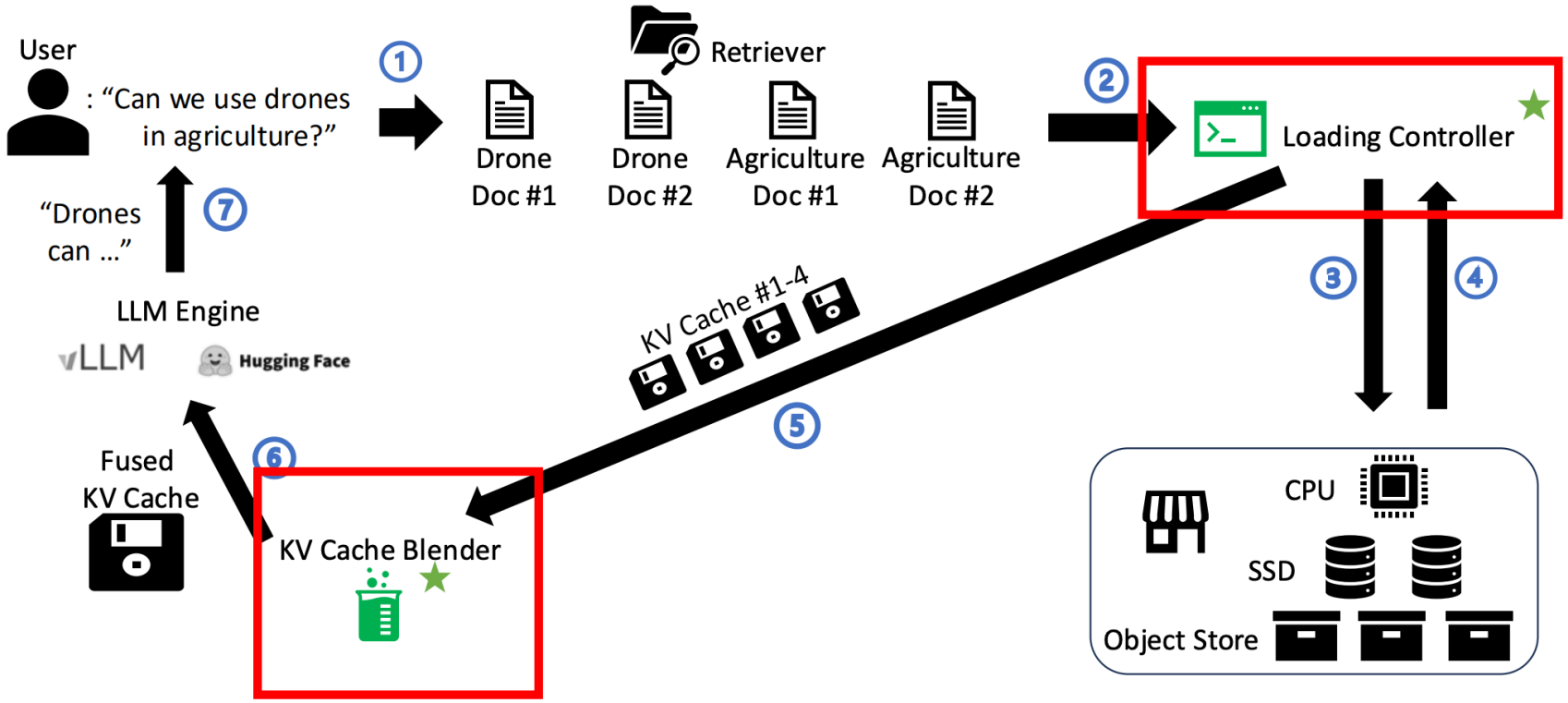


Figure 1. *Contrasting full KV recompute, prefix caching, full KV reuse, and CacheBlend's selective KV recompute.*

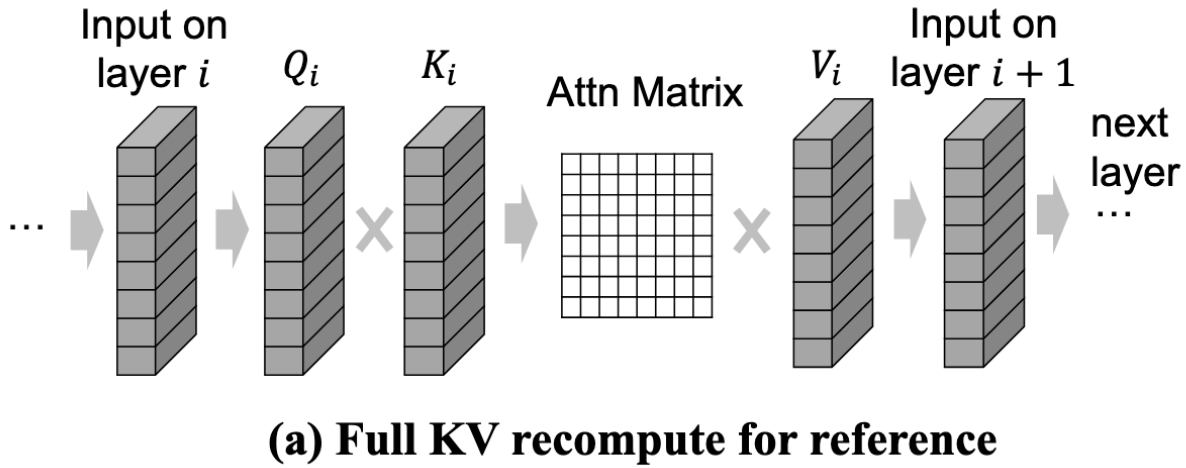
Overview -- Two key component

- **KV Cache Blender**
  - Selective recomputation to recover accuracy
- **Loading controller**
  - Pipeline Loading chunk KV Cache with recomputing
  - Find idealized recompute ratio according to storage device, LLM config, seqlen

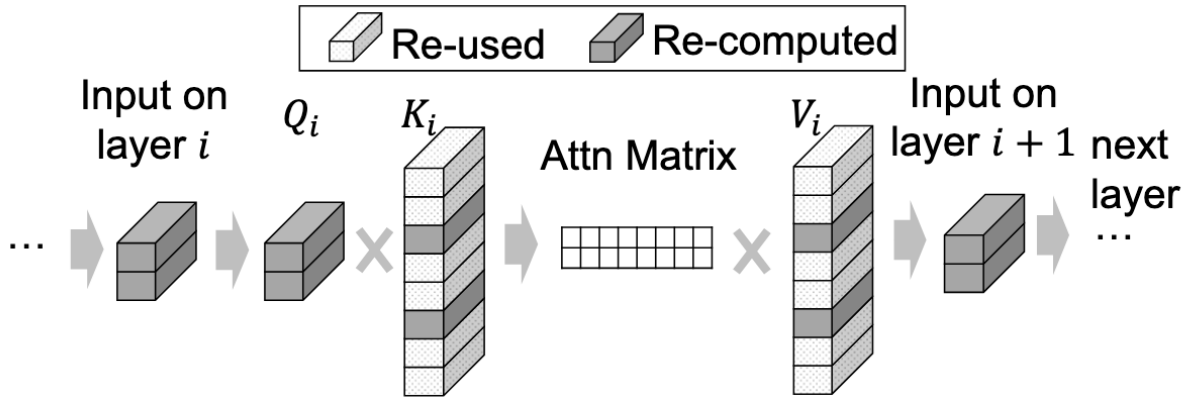


Design1

Selective recomputation



(a) Full KV recompute for reference



(b) Selective KV recompute on two selected tokens

(# total tokens)

↓ Reduced computation

(# selected tokens)

How to select tokens?

Design1

- Insight 1: The fraction of High-KV-Deviation tokens is small

Definition of KV deviation:

$$||\mathrm{KV}_{\mathrm{pre}} - \mathrm{KV}_{\mathrm{full}}||_2$$

$\mathrm{KV}_{\mathrm{pre}}$: **pre**-computed KV cache

$\mathrm{KV}_{\mathrm{full}}$: **Fully** recomputed KV cache

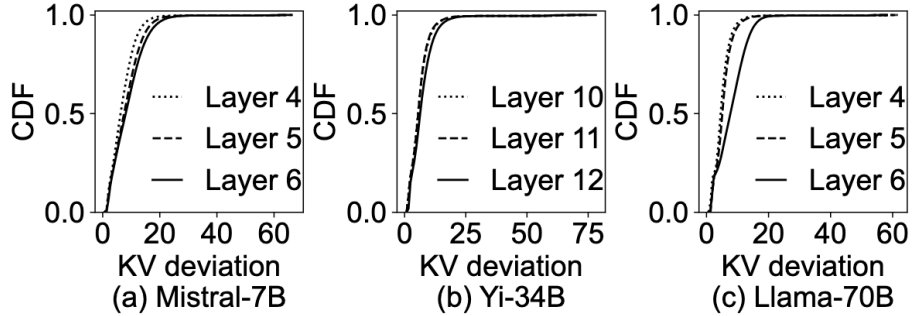


Figure 7. *Distribution of KV deviation of different tokens on one layer.*

- Insight 2: High-KV-Deviation tokens are similar across layers

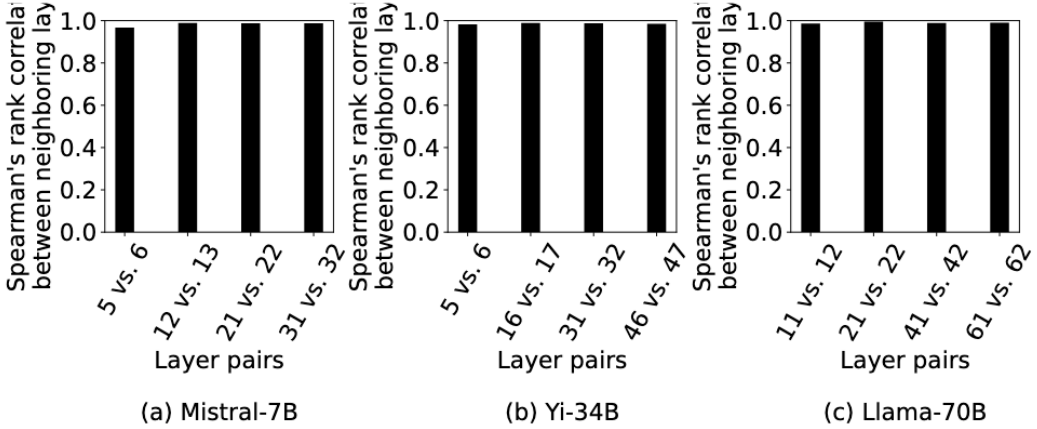
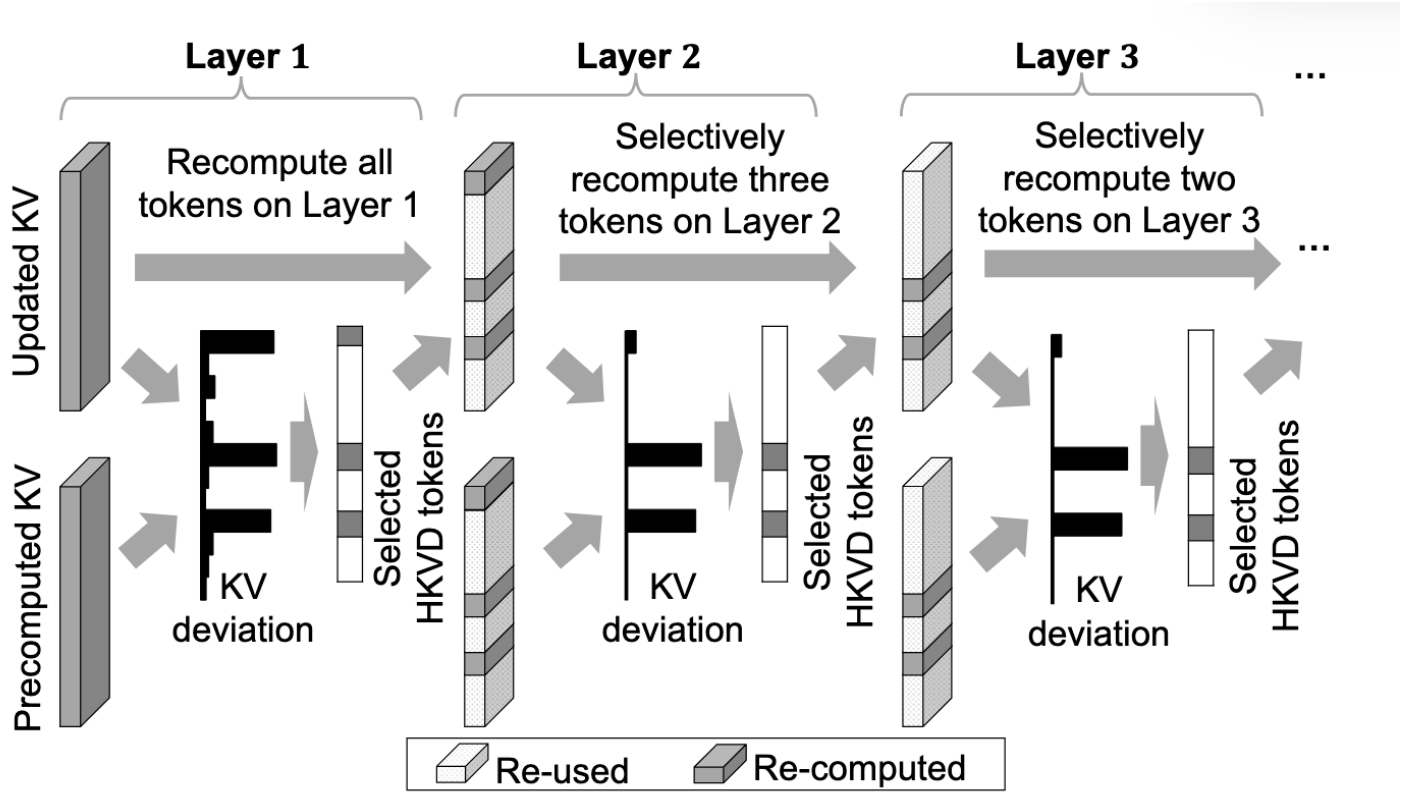


Figure 8. *Rank correlation of the KV deviation per token between two consecutive layers.*

Design1

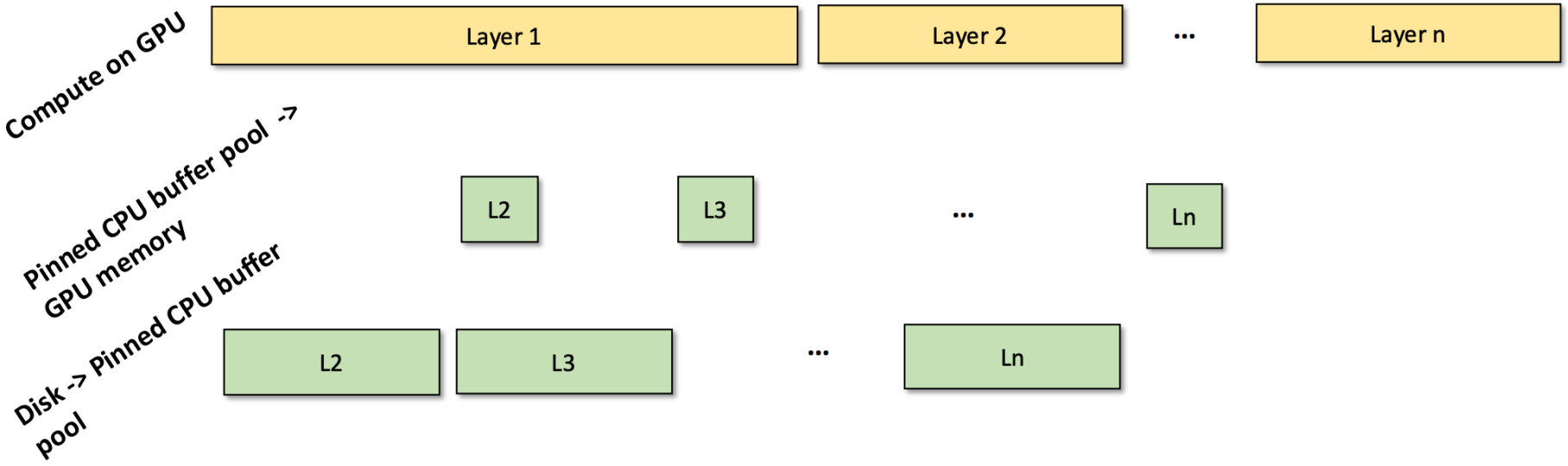
Selective recomputation



- **Selection Metric:** Deviation from pre-computed KV cache.
- Subset Constraint: $\text{Tokens}_{\text{Layer } i} \subseteq \text{Tokens}_{\text{Layer } i-1}$

Design2

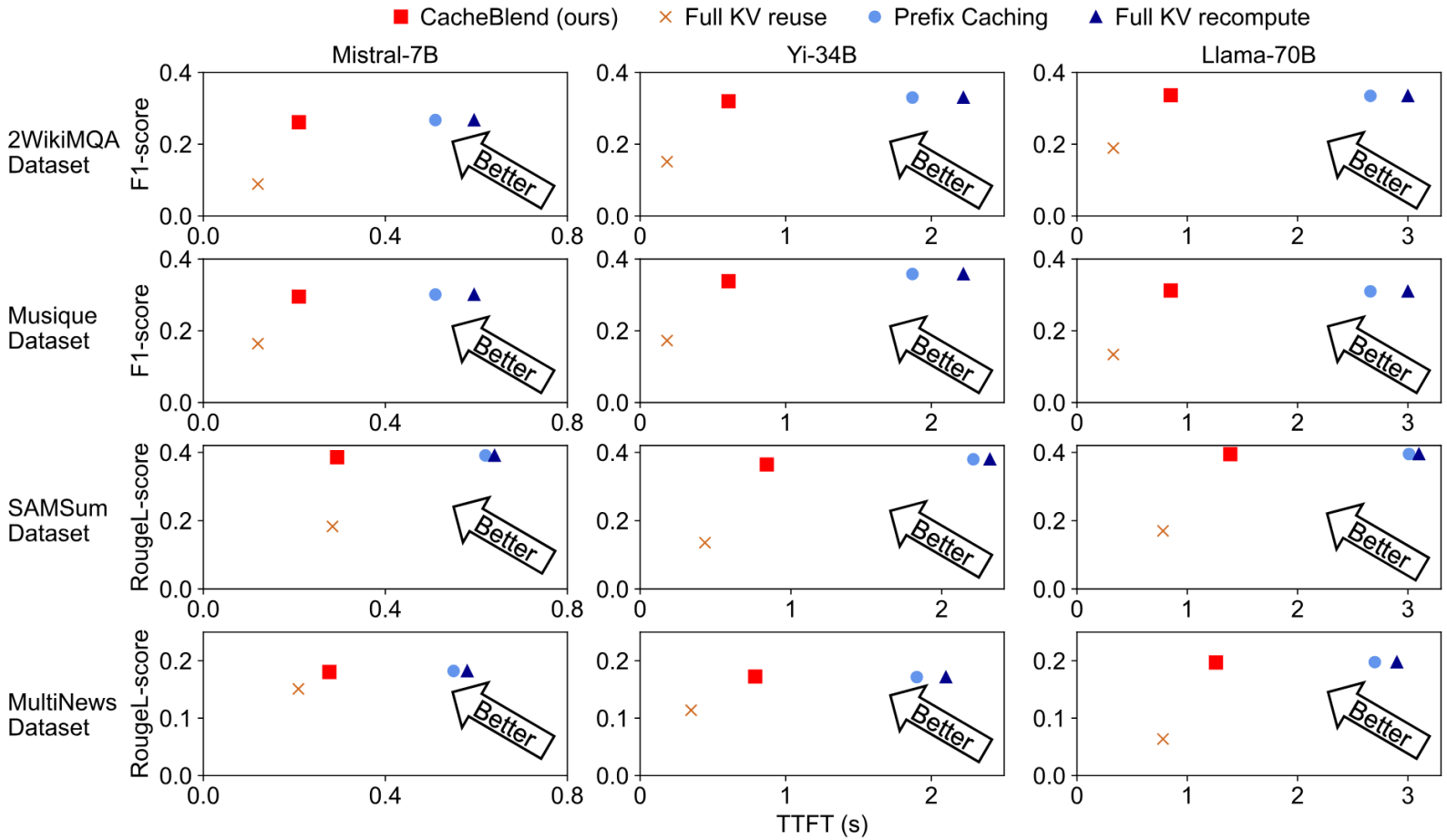
Loading Controller: Compute & load pipeline



Evaluation

Setup:

- Dataset: 2WikiMQA…
  - Top 6 chunks (512 tokens each)
- HW: GPU: 2x A40
- KV cache stored on 1TB NVME SSD
- Baselines: Full Recompute/ Full KV reuse/Caching Prefix



Conclusion: Best trade-off between Accuracy and TTFT.

FAST 2026

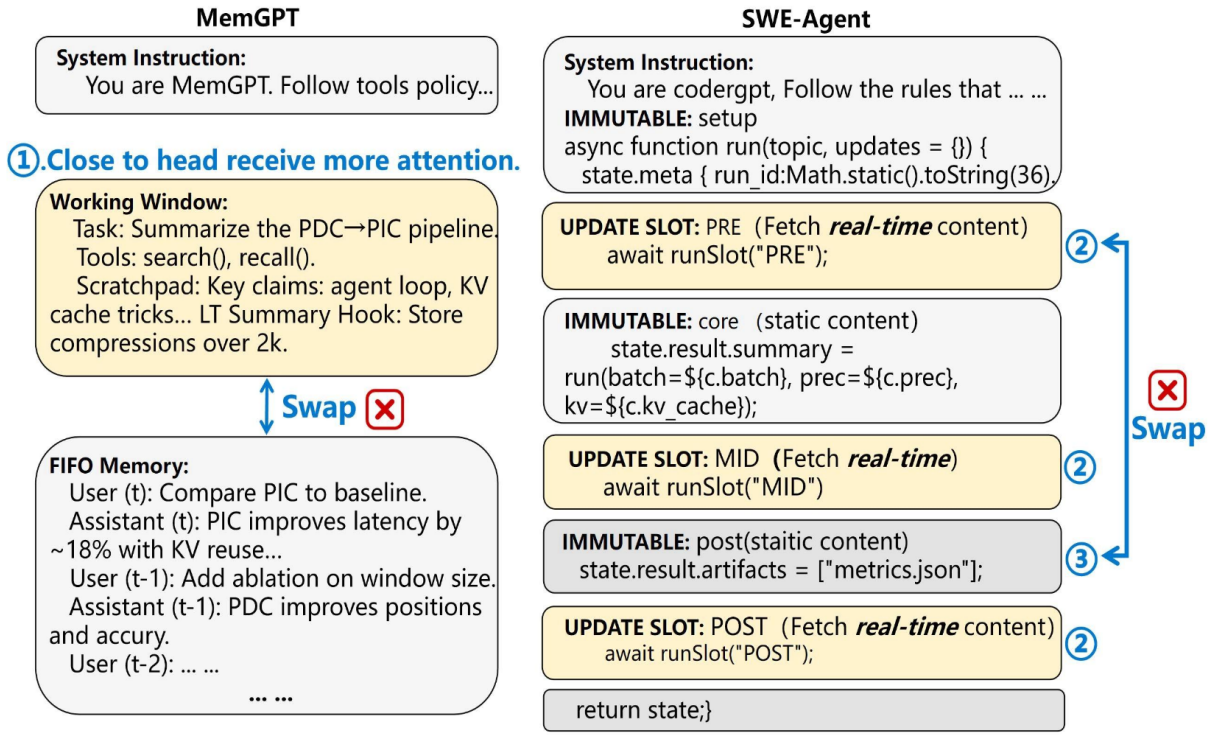
CacheSlide: Unlocking Cross Position-Aware KV Cache Reuse for Accelerating LLM Serving

Yang Liu and Yunfei Gu, *Shanghai Jiao Tong University;* Liqiang Zhang, *Jinan Inspur Data Technology Co., Ltd;* Chentao Wu, Guangtao Xue, Jie Li, and Minyi Guo, *Shanghai Jiao Tong University;* Junhao Hu, *Peking University;* Jie Meng, *Huawei Cloud*

- TL;DR:
  - Target scenario: Agent structure prompt
  - Core idea:

Agent prompt structure

- immutable segments remain invariant across multiple inferences
- Interleaved immutable & updated segments, constant relative positions in immutable segments



Iterative Prompt Evolution via Update Slots:

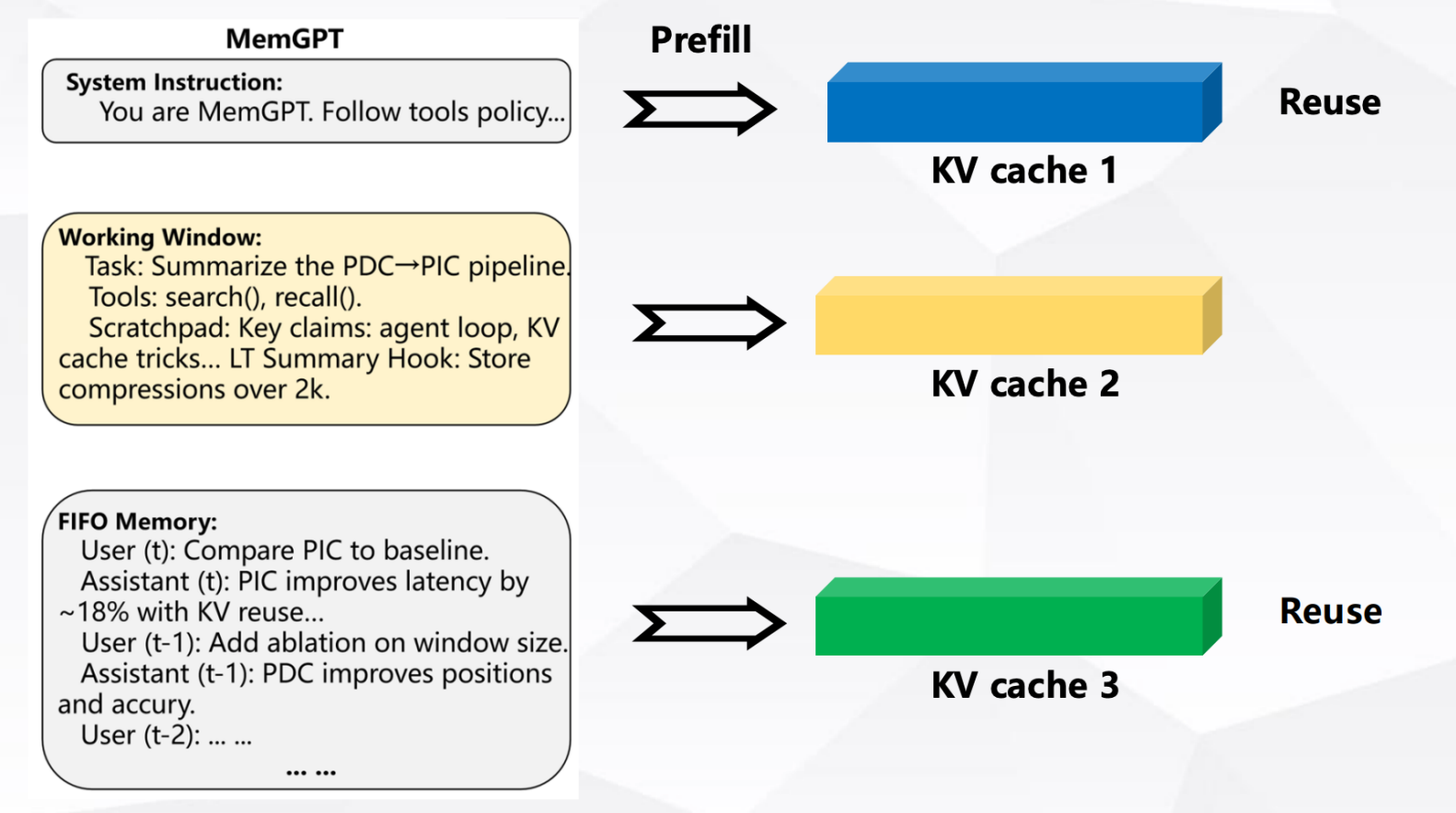
- LLM(prompt1) -> Respone1
- Update(prompt1, Respone1) -> prompt2
- LLM(prrompt2) -> Respone2
- Update(prompt2, Respone2)-> prompt3

…

Figure 2: Examples of multi-segment and single-segment updates.

Motivation -- Reuse KV cache in agents

- Cache and reuse KV states of unchanged prompt segments to significantly reduces compute overhead & TTFT

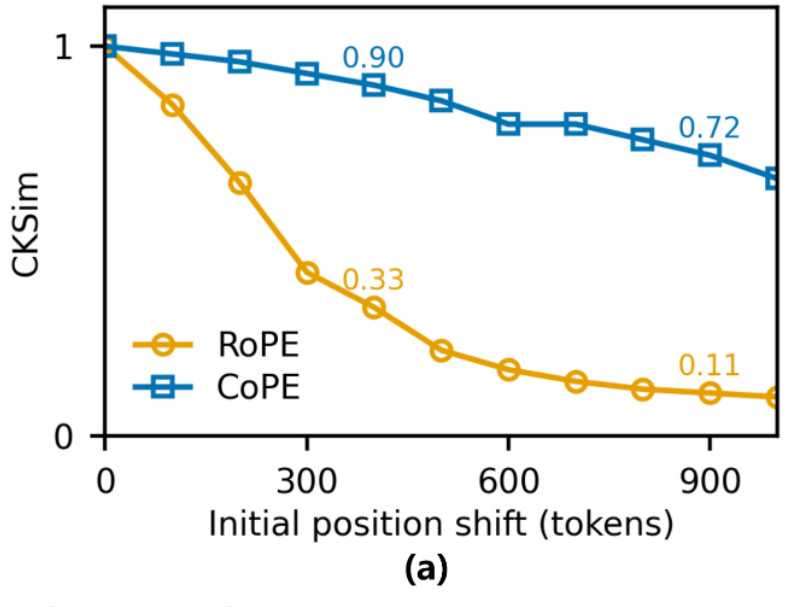


Motivation– limitation of PIC

- Inaccuracy:
  - Loss position information (Positionally Misaligned KV Drift)
  - different attention heads attend to different tokens, recomputing only a subset can make generation quality unstable

Position info importance experiment:

- **Baseline:** Encode MemHistory ->**KV_0**
- **Positional Shift:** position embedding shift length i(100, 200, 300…)-> **KV_i**
- **Metric:** Calculate **CKSim** between KV_i and KV_0



Overview

- three modules:
  - CCPE assigns the cached KV states the most frequent CoPE encoding
  - WCA selectively recomputes a subset of tokens in the KV cache
  - SLIDE reduces KV-cache overflow and the latency caused by inter-layer read–write coupling during prefill.

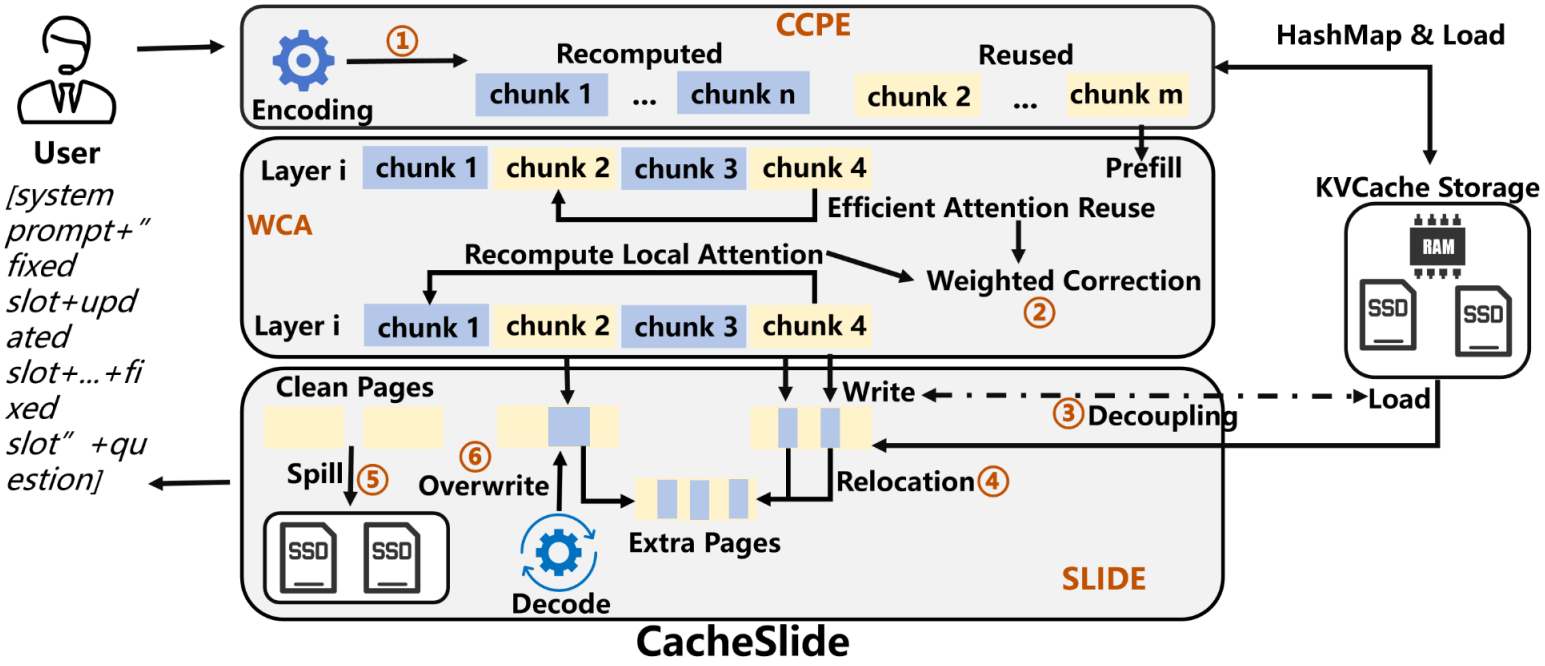


Figure 6: Block diagram of proposed CacheSlide system workflow

PS:
Recomputed: updated segment
Reused: immutable segment

Chunked Contextual Position Encoding(CCPE)

- **Offline Phase:**
  - Pretrain CoPE(low positional sensitivity) on a specific task
  - Profile frequent position patterns to **assign fixed positional embedding** to reusable chunks
- **Online Phase (Inference):**
  - Split inputs into **Reused Chunks** and **Recomputed Chunks** based on prompt templates

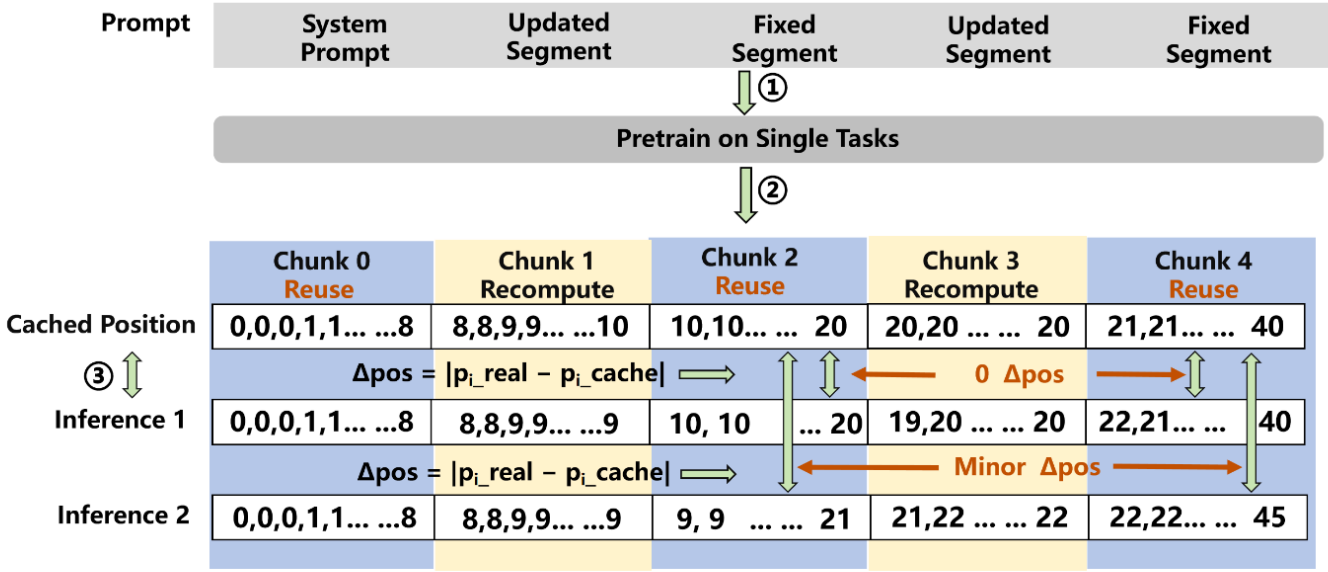


Figure 7: Process of Chunked Contextual Position Encoding.

Weighted Correction Attention(WCA)

- **Initial selection:** Recompute Layer 1 to measure KV drift, selecting the top-k most deviated tokens (S_k).
- **Dynamic update:** Periodically refine S_k using CKSim in deeper layers to recovers critical cross-attention with minimal compute overhead.

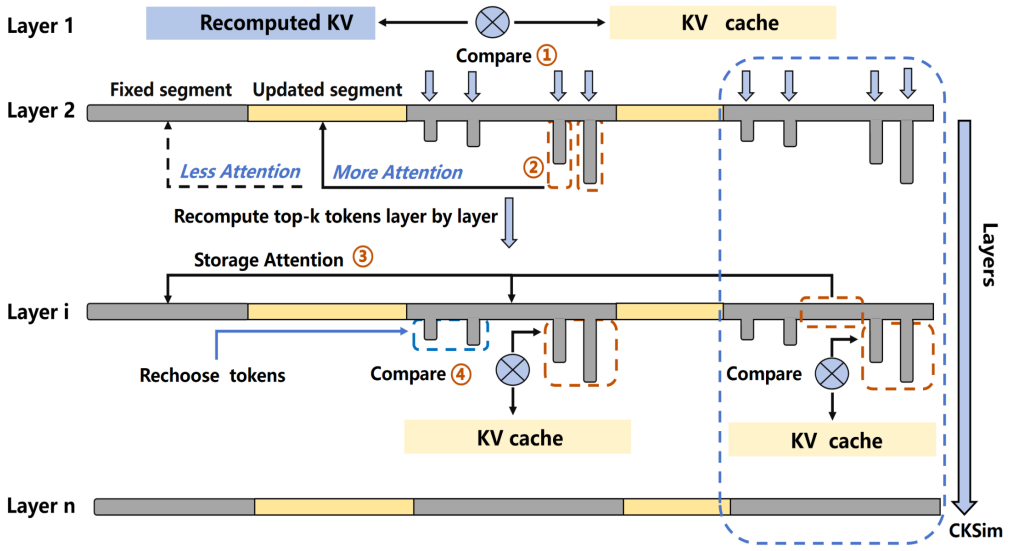


Figure 8: Scheme of Weighted Correction Attention.

SLIDE (KV cache Management)

- **Load-write decoupling: pipeline updated kv write and kv cache load from ssd**
- **Dirty-aware eviction:**Prioritizes clean pages, then evicts dirty pages by descending token count to minimize **WAF**

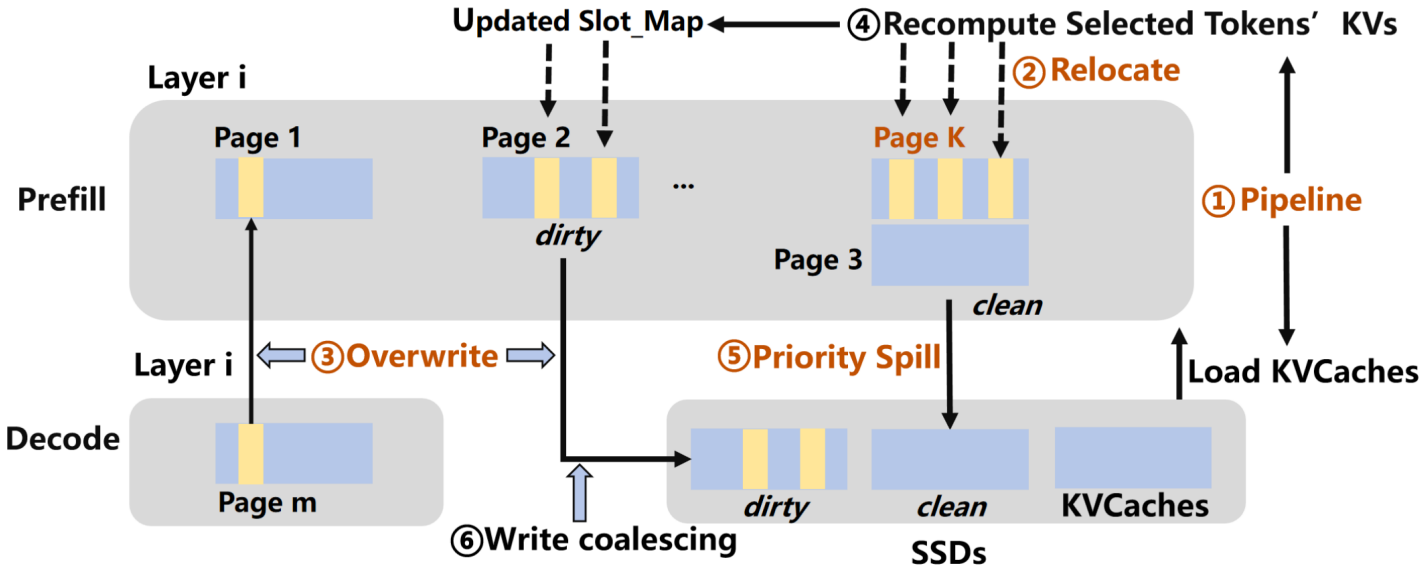


Figure 9: Decoupled Load–Write and Spill-Aware KV cache Management in SLIDE.

Evaluation

Experiment Setup

Item	Implementation Environment
Inference Engine	vLLM 0.8.5
Models	Mistral 7B, MPT 30B, Llama-3 70B
GPU	A100
Storage	500 GB DRAM, 2 TB NVMe SSDs
Interconnect	PCIE Gen4

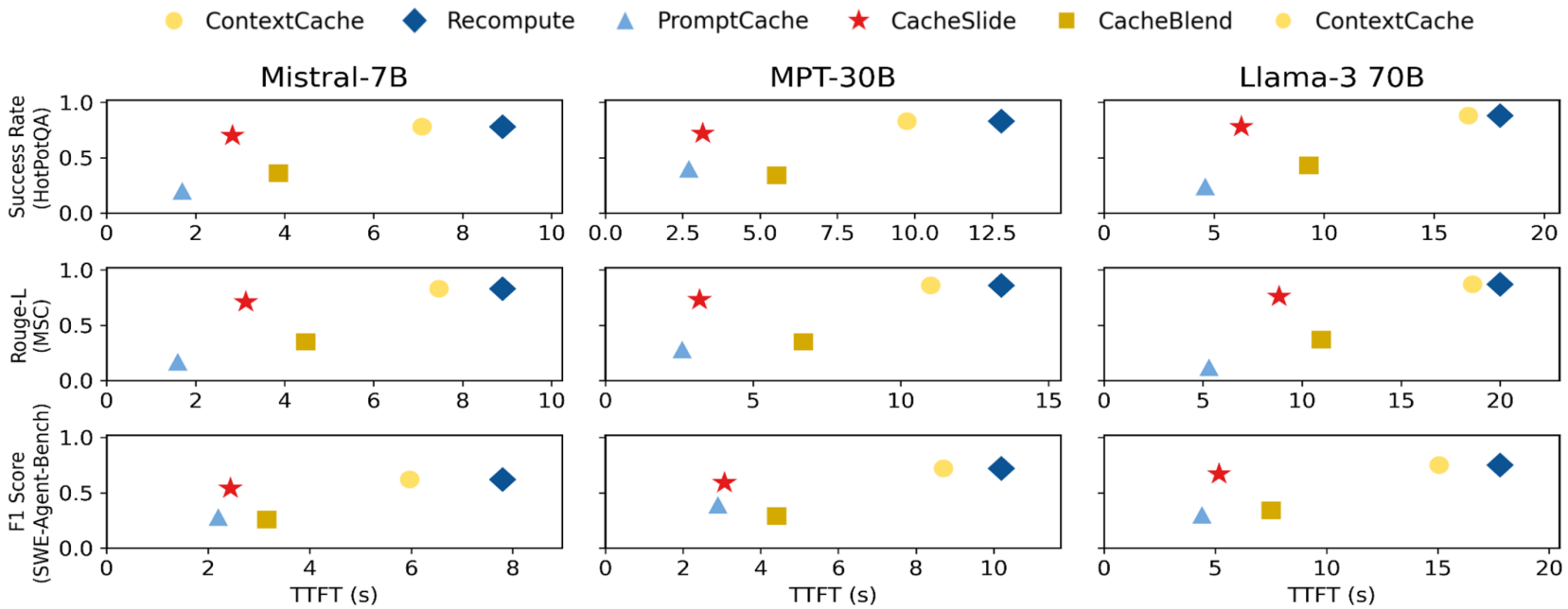
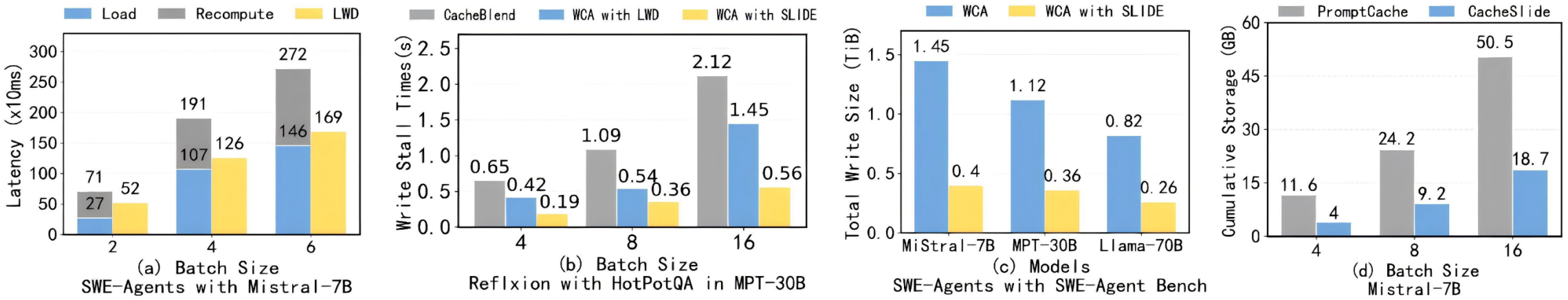


Figure 10: Across three models and datasets, CacheSlide achieves a 2.4-3.3× reduction in TTFT with negligible accuracy loss compared to ContextCache. Compared to CacheBlend, CacheSlide achieves a 1.21-2.11× reduction in TTFT and a 1.97-2.28× improvement in accuracy. Compared to PromptCache, CacheSlide achieves a 1.12-2.45× reduction in TTFT and a 1.41-3.95× improvement in accuracy. Other systems' metrics can be inferred from these key results.

- **vs. CacheBlend (PIC):** TTFT reduced by 1.21x - 2.11x, Accuracy improved by 1.97x - 2.28x
- **vs. PromptCache (PDC):** TTFT reduced by 1.12x - 2.45x, Accuracy improved by 1.41x - 3.95x
- **Conclusion:** The **only** solution that simultaneously dominates in both efficiency and accuracy.

Evaluation

SLIDE Ablation Study



- **Load-Write Decoupling (LWD):** reduces layer-wise parallel latency by 26.7–51.5%
- **Dirty-Page Eviction:** decreases write stalls by 66.9–73.5%
- **Storage Efficiency:** SSD WAF reduced by 3.11x - 3.62x.

Summary

Three KV cache reuse Paradigms in LLM serving

Caching Paradigm	Target Scenario	Position Constraint	Accuracy (Quality)	Speed (TTFT)	Recompute Overhead
PDC (Prefix Cache)	Multi-turn Chat	Strict Prefix	**High** (Lossless)	**Slowest** (Low hit rate ⟶ Full recompute)	Zero
PIC (CacheBlend)	RAG	Position Agnostic	**Low** (Severe PMKD & misalignment)	Middle	Medium (~15% fixed ratio)
RPDC (CacheSlide)	Agent Serving	Relative Order Preserved	Near-lossless	**Fast**	Low (Dynamic Top-k subset)

Summary

1. Why CacheSlide > CacheBlend (Accuracy)

- **Positional Awareness:** Encodes immutable segments using their most frequent task positions, mitigating position drift.
- **Inter-Segment Context:** Preserves cross-attention *between* fixed segments (CacheBlend treats segments as isolated modules, keeping only intra-segment attention).

2. Speed Comparison

- Prefix Cache (Slowest): Rigid exact-match constraint leads to near-zero hit rates in dynamic agent prompts, constantly degrading to full recomputation.
- CacheSlide (Fastest): minimizie recompute overhead and I/O system optimization over CacheBlend